Reinforcement Learning

Deep Q-Learning & Advanced Policy Gradients

SJTU

Outline

Part I: Making Q-Learning Work

- Recap: Q-learning issues
- Replay buffers
- Target networks & DQN
- Double Q-learning
- Multi-step returns
- Continuous actions & DDPG

Part II: Advanced Policy Gradients

- Policy gradient as policy iteration
- Bounding the distribution change
- Trust regions & KL constraints
- Natural gradient
- TRPO & PPO

Part I: Making Q-Learning Work

From online Q-learning to DQN and beyond

Recap: Q-Learning

Full fitted Q-iteration:

1. Collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)\}$ using some policy
2. Set $\mathbf{y}_i \leftarrow r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)$
3. Set $\phi \leftarrow \arg\min_\phi \frac{1}{2} \sum_i \|Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{y}_i\|^2$

Steps 2–3 repeat K times before collecting more data.

Online Q-learning (N=1, K=1, one gradient step):

1. Take action $\mathbf{a}_i$, observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$
2. $\mathbf{y}_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)$
3. $\phi \leftarrow \phi - \alpha \frac{dQ_\phi}{d\phi}(\mathbf{s}_i, \mathbf{a}_i)(Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{y}_i)$

Two Problems with Online Q-Learning

- Sequential samples $(\mathbf{s}_t, \mathbf{s}_{t+1}, \ldots)$ are **strongly correlated** $\rightarrow$ violates i.i.d. SGD assumption
- Q-learning is **not gradient descent**: no gradient through target $\mathbf{y}_i$ (semi-gradient)

Replay Buffers

Key idea: store all transitions in a buffer $\mathcal{B}$; sample i.i.d. mini-batches.

Q-learning with replay buffer:

1. Sample a batch $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$ from $\mathcal{B}$
2. $\phi \leftarrow \phi - \alpha \sum_i \frac{dQ_\phi}{d\phi}(\mathbf{s}_i, \mathbf{a}_i)(Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - [r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)])$

Benefits

- Samples are no longer correlated
- Multiple samples in batch $\rightarrow$ low-variance gradient
- Off-policy: any policy's data can be reused

Remaining Issue

- Target $\mathbf{y}_i = r + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}', \mathbf{a}')$ still changes with ϕ
- Not stable regression: moving target!
- Need target networks to fix this

Note: replay buffer is a special case of fitted Q-iteration with K=1 and one gradient step.

Target Networks

Problem: in Q-learning with replay buffer, target $\mathbf{y}_i$ changes after every gradient step on ϕ.
Solution: use a **separate target network** $Q_{\phi'}$ that is updated slowly.

Q-learning with replay buffer and target network:

1. Save target network parameters: $\phi' \leftarrow \phi$
2. Collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)\}$, add to $\mathcal{B}$
3. Sample batch from $\mathcal{B}$; compute $\mathbf{y}_j = r_j + \gamma \max_{\mathbf{a}'} Q_{\phi'}(\mathbf{s}'_j, \mathbf{a}'_j)$ using *target* network
4. $\phi \leftarrow \phi - \alpha \sum_j \frac{dQ_\phi}{d\phi}(\mathbf{s}_j, \mathbf{a}_j)(Q_\phi(\mathbf{s}_j, \mathbf{a}_j) - \mathbf{y}_j)$
5. Update ϕ': copy ϕ every N steps

Steps 2–4 repeat; targets are **fixed** within the inner loop $\rightarrow$ supervised regression!

Polyak Averaging (Popular Alternative)

Instead of hard copy every N steps:

$$\phi' \leftarrow \tau\phi' + (1-\tau)\phi, \quad \tau = 0.999$$

Provides smoother, more consistent target lag.

"Classic" Deep Q-Learning: DQN

DQN (Mnih et al., 2013/2015): N=1, K=1 in the general framework.

1. Take action $\mathbf{a}_i$ (e.g., ϵ-greedy), observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$, add to $\mathcal{B}$
2. Sample mini-batch $\{\mathbf{s}_j, \mathbf{a}_j, \mathbf{s}'_j, r_j\}$ from $\mathcal{B}$ uniformly
3. Compute $\mathbf{y}_j = r_j + \gamma \max_{\mathbf{a}'_j} Q_{\phi'}(\mathbf{s}'_j, \mathbf{a}'_j)$ using *target* network $Q_{\phi'}$
4. $\phi \leftarrow \phi - \alpha \sum_j \frac{dQ_\phi}{d\phi}(\mathbf{s}_j, \mathbf{a}_j)(Q_\phi(\mathbf{s}_j, \mathbf{a}_j) - \mathbf{y}_j)$
5. Update ϕ': copy ϕ every N steps

Three Key Ingredients

- **Replay buffer**: break sample correlations
- **Target network**: stabilize targets
- **ϵ-greedy**: explore while exploiting

A General View: 3 Processes

- **Process 1:** data collection (environment $\rightarrow \mathcal{B}$)
- **Process 2:** target update ($\phi \rightarrow \phi'$)
- **Process 3:** Q-function regression (SGD on ϕ)

DQN: processes 1 & 3 fast, process 2 slow.

Double Q-Learning

The Overestimation Problem

Target: $\mathbf{y}_j = r_j + \gamma \max_{\mathbf{a}'} Q_{\phi'}(\mathbf{s}'_j, \mathbf{a}')$. Since $Q_{\phi'}$ is noisy:

$$\mathbb{E}[\max(X_1, X_2)] \geq \max(\mathbb{E}[X_1], \mathbb{E}[X_2])$$

The max operator systematically **overestimates** Q-values because the same network selects the action *and* evaluates its value.

Standard Q-Learning

$$\mathbf{y} = r + \gamma Q_{\phi'}(\mathbf{s}', \arg\max_{\mathbf{a}'} Q_{\phi'}(\mathbf{s}', \mathbf{a}'))$$

Same network for selection *and* evaluation.

Double Q-Learning

$$\mathbf{y} = r + \gamma Q_{\phi'}(\mathbf{s}', \arg\max_{\mathbf{a}'} Q_{\phi}(\mathbf{s}', \mathbf{a}'))$$

Current network Q_ϕ selects action;
target network $Q_{\phi'}$ evaluates value.

If noise in Q_ϕ and $Q_{\phi'}$ is decorrelated, overestimation bias is eliminated. In practice, Double DQN significantly reduces value overestimation and improves performance.

Multi-Step Returns

Standard target (N=1):

$$\mathbf{y}_{j,t} = r_{j,t} + \gamma \max_{\mathbf{a}'} Q_{\phi'}(\mathbf{s}_{j,t+1}, \mathbf{a}')$$

***N*-step return target:**

$$\mathbf{y}_{j,t} = \sum_{t'=t}^{t+N-1} \gamma^{t'-t} r_{j,t'} + \gamma^N \max_{\mathbf{a}'} Q_{\phi'}(\mathbf{s}_{j,t+N}, \mathbf{a}')$$

Benefits

- Less biased when Q-values are inaccurate
- Faster learning, especially early on

Caveat

N-step targets estimate $Q^\pi(\mathbf{s}_{j,t}, \mathbf{a}_{j,t})$ for the *data-collection* policy π.
Only correct when learning on-policy (not an issue when N=1).

How to fix?

- **Ignore it** (often works well)
- **Cut the trace**: dynamically choose N
- **Importance sampling**

Recall: actor-critic with MC target = no bias, high variance. N=1 Q-learning target = max bias, min variance. N-step = interpolation.

Q-Learning with Continuous Actions

Problem: policy $\pi(\mathbf{a}|\mathbf{s}) = \delta(\mathbf{a} = \arg\max_{\mathbf{a}} Q_\phi)$ and target $\mathbf{y} = r + \gamma \max_{\mathbf{a}'} Q_{\phi'}$ both require max over continuous $\mathbf{a}$.

Opt. 1: Stochastic

$$\max_{\mathbf{a}} Q \approx \max_{i} Q(\mathbf{s}, \mathbf{a}_i)$$

Sample $\mathbf{a}_1, \ldots, \mathbf{a}_N$ randomly.
CEM, CMA-ES ($\leq$40 dims).

Opt. 2: NAF

$$Q_\phi = -\tfrac{1}{2}(\mathbf{a}-\mu)^T P(\mathbf{a}-\mu) + V$$

$\arg\max Q = \mu_\phi(\mathbf{s})$.
+ Efficient. – Limited expressiveness.

Opt. 3: DDPG

Learn $\mu_\theta \approx \arg\max_{\mathbf{a}} Q_\phi$:

$$\theta \leftarrow \arg\max_{\theta} Q_\phi(\mathbf{s}, \mu_\theta)$$

$$\frac{dQ}{d\theta} = \frac{d\mathbf{a}}{d\theta}\frac{dQ}{d\mathbf{a}}$$

"Deterministic actor-critic."

DDPG (Lillicrap et al., 2016): actor $\mu_\theta(\mathbf{s})$ approximates argmax. Target: $\mathbf{y}_j = r_j + \gamma Q_{\phi'}(\mathbf{s}'_j, \mu_\theta(\mathbf{s}'_j))$.

Part II: Advanced Policy Gradients

From policy iteration theory to trust regions and natural gradients

Policy Gradient as Policy Iteration

Recall: actor-critic estimates $\hat{A}^{\pi}(\mathbf{s}_t, \mathbf{a}_t)$ and uses it to improve π.
This is exactly **policy iteration**:

1. Evaluate $A^{\pi}(\mathbf{s}, \mathbf{a})$
2. Set $\pi \leftarrow \pi'$ (improved policy)

Key Identity

$$J(\theta') - J(\theta) = \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t) \right]$$

The improvement equals the expected discounted advantage of the *new* policy $\pi_{\theta'}$, evaluated using the *old* advantage A^{π_θ}.

Goal: maximize this expression. But the expectation is under $p_{\theta'}(\tau)$ — the new trajectory distribution — which we don't have samples from!

Idea: use importance sampling to convert to old-policy samples:

$$J(\theta') - J(\theta) = \sum_t \mathbb{E}_{\mathbf{s}_t \sim p_{\theta'}} \left[\mathbb{E}_{\mathbf{a}_t \sim \pi_\theta} \left[\frac{\pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t | \mathbf{s}_t)} \gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t) \right] \right]$$

Ignoring the Distribution Mismatch

We have:

$$J(\theta') - J(\theta) = \sum_t \mathbb{E}_{\mathbf{s}_t \sim p_{\theta'}} \left[\mathbb{E}_{\mathbf{a}_t \sim \pi_\theta} \left[\frac{\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)} \gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t) \right] \right]$$

We want: replace $p_{\theta'}(\mathbf{s}_t)$ with $p_\theta(\mathbf{s}_t)$ (which we can sample from).
Define the **surrogate objective**:

$$\bar{A}(\theta') = \sum_t \mathbb{E}_{\mathbf{s}_t \sim p_\theta} \left[\mathbb{E}_{\mathbf{a}_t \sim \pi_\theta} \left[\frac{\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)} \gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t) \right] \right]$$

When Is This Approximation Valid?

$p_\theta(\mathbf{s}_t)$ is *close* to $p_{\theta'}(\mathbf{s}_t)$ when π_θ is *close* to $\pi_{\theta'}$.
$\Rightarrow$ If we constrain the **policy change** to be small, the surrogate is a good approximation.

Key result: $\theta' \leftarrow \arg\max_{\theta'} \bar{A}(\theta')$ subject to $|\pi_{\theta'}(\mathbf{a}|\mathbf{s}) - \pi_\theta(\mathbf{a}|\mathbf{s})| \leq \epsilon$ guarantees improvement $J(\theta') - J(\theta) \geq 0$ for small enough ϵ.

Bounding the Distribution Change

Claim: $|p_{\theta'}(\mathbf{s}_t) - p_\theta(\mathbf{s}_t)| \leq 2\epsilon t$ when $|\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t) - \pi_\theta(\mathbf{a}_t|\mathbf{s}_t)| \leq \epsilon$.

Proof sketch (deterministic case): assume $\mathbf{a}_t = \pi_\theta(\mathbf{s}_t)$.

$\pi_{\theta'}$ takes a different action with probability $\leq \epsilon$ at each step.

$$p_{\theta'}(\mathbf{s}_t) = \underbrace{(1-\epsilon)^t}_{\text{no mistakes}} p_\theta(\mathbf{s}_t) + (1-(1-\epsilon)^t)\, p_{\text{mistake}}(\mathbf{s}_t)$$

$$|p_{\theta'}(\mathbf{s}_t) - p_\theta(\mathbf{s}_t)| \leq 2(1-(1-\epsilon)^t) \leq 2\epsilon t$$

Using $(1-\epsilon)^t \geq 1 - \epsilon t$ for $\epsilon \in [0, 1]$.

General case: extends to stochastic π_θ via coupling lemma.

Bounding the Objective

$$J(\theta') - J(\theta) \geq \bar{A}(\theta') - \sum_t 2\epsilon t \cdot C, \quad C = O\left(\frac{r_{\max}}{1-\gamma}\right)$$

Maximizing $\bar{A}(\theta')$ while constraining ϵ small $\rightarrow$ **guaranteed improvement**!

Trust Regions: KL Divergence Constraint

A more convenient bound: use KL divergence instead of $|\pi_{\theta'} - \pi_\theta|$:

$$|\pi_{\theta'}(\mathbf{a}|\mathbf{s}) - \pi_\theta(\mathbf{a}|\mathbf{s})| \leq \sqrt{\tfrac{1}{2} D_{\mathrm{KL}}(\pi_{\theta'}(\mathbf{a}|\mathbf{s}) \| \pi_\theta(\mathbf{a}|\mathbf{s}))}$$

Constrained optimization problem:

$$\theta' \leftarrow \arg\max_{\theta'} \bar{A}(\theta') \quad \text{s.t.} \quad D_{\mathrm{KL}}(\pi_{\theta'}(\mathbf{a}|\mathbf{s}) \| \pi_\theta(\mathbf{a}|\mathbf{s})) \leq \epsilon$$

Dual Gradient Descent

Lagrangian:

$$\mathcal{L}(\theta', \lambda) = \bar{A}(\theta') - \lambda(D_{\mathrm{KL}} - \epsilon)$$

1. Maximize $\mathcal{L}$ w.r.t. θ' (few grad steps)
2. $\lambda \leftarrow \lambda + \alpha(D_{\mathrm{KL}} - \epsilon)$

Raise λ if constraint violated.

Why KL?

- KL bounds state marginal difference
- KL has a nice 2nd-order Taylor expansion
- Leads to natural gradient and TRPO

Natural Gradient

Linearize the objective $\bar{A}(\theta')$:

$$\theta' \leftarrow \arg\max_{\theta'} \nabla_\theta \bar{A}(\theta)^T(\theta' - \theta) \quad \text{s.t.} \quad D_{\mathrm{KL}}(\pi_{\theta'} \| \pi_\theta) \leq \epsilon$$

Note: $\nabla_\theta \bar{A}(\theta) = \nabla_\theta J(\theta)$ — the standard policy gradient!

Regular gradient ascent constrains $\|\theta' - \theta\|^2 \leq \epsilon$. But a small change in θ may cause a *large* change in $\pi_\theta(\mathbf{a}|\mathbf{s})$ (and vice versa).

2nd-order Taylor expansion of KL:

$$D_{\mathrm{KL}}(\pi_{\theta'} \| \pi_\theta) \approx \tfrac{1}{2}(\theta' - \theta)^T \mathbf{F} (\theta' - \theta)$$

where $\mathbf{F} = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(\mathbf{a}|\mathbf{s}) \nabla_\theta \log \pi_\theta(\mathbf{a}|\mathbf{s})^T]$ is the **Fisher information matrix**.

Natural Gradient Update

$$\theta' = \theta + \alpha \mathbf{F}^{-1} \nabla_\theta J(\theta), \qquad \alpha = \sqrt{\frac{2\epsilon}{\nabla_\theta J(\theta)^T \mathbf{F}^{-1} \nabla_\theta J(\theta)}}$$

Steps in *distribution space* rather than parameter space $\rightarrow$ better-conditioned optimization.

TRPO & PPO

TRPO (Schulman et al., 2015)

- Directly maximize $\bar{A}(\theta')$ with KL constraint
- Use conjugate gradients for $\mathbf{F}^{-1}\mathbf{g}$ (no explicit $\mathbf{F}$)
- Line search to ensure improvement
- Theoretically principled; monotonic improvement guarantee
- Complex implementation

PPO (Schulman et al., 2017)

- Simpler surrogate: clip the IS ratio

$$L^{\text{CLIP}} = \mathbb{E}\left[\min\left(\frac{\pi_{\theta'}}{\pi_\theta}\hat{A},\ \text{clip}\left(\frac{\pi_{\theta'}}{\pi_\theta}, 1\pm\epsilon\right)\hat{A}\right)\right]$$

- No KL constraint; clipping prevents large updates
- Easy to implement; works with standard SGD
- The most widely used PG algorithm today

Practical Recommendations

- **Natural policy gradient**: good general stabilizer for PG training
- **TRPO**: when you need monotonic improvement guarantees
- **PPO**: default choice for most applications (robotics, games, LLM alignment)

Summary

Deep Q-Learning

- **Replay buffer**: decorrelate samples
- **Target network**: stabilize regression targets
- **DQN** = replay + target + ϵ-greedy
- **Double Q-learning**: fix overestimation bias
- **Multi-step returns**: bias–variance trade-off
- **Continuous actions**: DDPG (learned arg max)

Advanced Policy Gradients

- PG = policy iteration (optimize advantage)
- State marginal mismatch $\rightarrow$ constrained opt.
- KL divergence bounds distribution change
- **Natural gradient**: $\mathbf{F}^{-1}\nabla J$ (distribution-aware steps)
- **TRPO**: conjugate gradients + line search
- **PPO**: clipped surrogate (simple, effective)

The Deep RL Landscape

- **Value-based** (DQN family): off-policy, discrete actions, sample-efficient
- **Policy gradient** (PPO family): on-policy, continuous actions, stable
- **Actor-critic** (DDPG, SAC): best of both worlds — off-policy + continuous